

Spark Resource Managers

源码深度分析

Kubernetes + YARN 资源管理器架构与实现

模块	核心文件数	代码规模	关键设计
Kubernetes	~40 Scala	~6000行	Feature Step链+事件驱动Pod管理
YARN	~25 Scala+4 Java	~7000行	AM双模式+三级本地性匹配

基于 Apache Spark v3.3.1 源码分析

目录

第一章 模块架构总览

第二章 Kubernetes 资源管理器

2.1 Driver提交流程

2.2 Feature Step链式配置

2.3 Executor Pod事件驱动模型

2.4 Pod分配器策略

2.5 核心类继承体系

2.6 Shuffle与PVC复用

2.7 插件扩展

第三章 YARN 资源管理器

3.1 Client Mode完整流程

3.2 Cluster Mode完整流程

3.3 YarnAllocator资源分配

3.4 本地性匹配策略

3.5 核心类继承体系

3.6 AM与RM交互

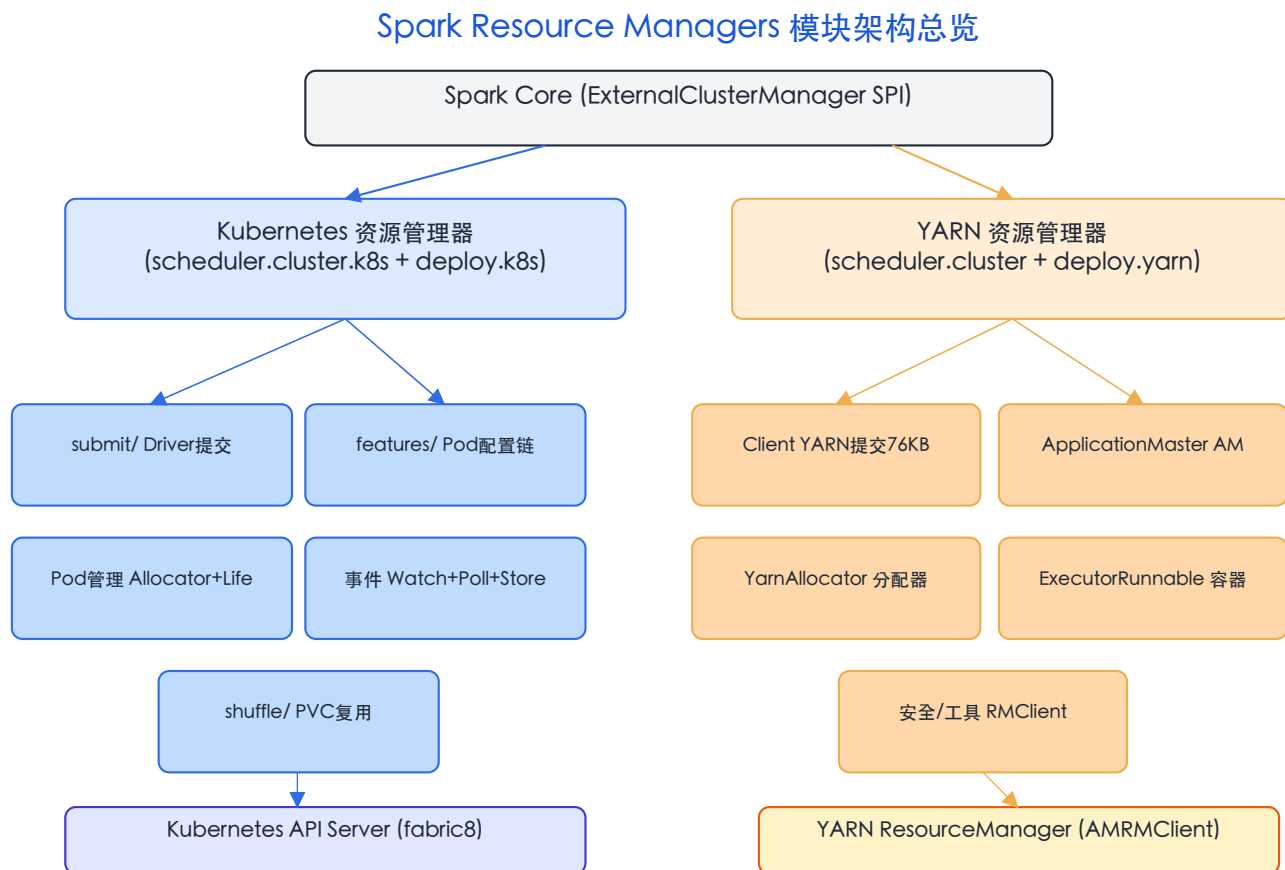
第四章 K8s vs YARN对比

第五章 核心设计精髓

第一章 模块架构总览

Spark Resource Managers 模块是 Spark 与外部集群管理系统的桥梁，通过 ExternalClusterManager SPI 接口实现可插拔的资源管理。当前版本支持 Kubernetes 和 YARN 两大资源管理器，分别适用于容器化云原生和 Hadoop 生态部署场景。

图 1-1：模块架构总览图



1.1 共同架构模式

两个资源管理器虽面向不同底层平台，但共享相同的 Spark 调度框架：

架构层次	Kubernetes实现	YARN实现
SPI入口	KubernetesClusterManager	YarnClusterManager
调度后端	K8sClusterSchedulerBackend (继承CoarseGrainedSchedulerBackend)	YarnSchedulerBackend (Client/Cluster两种子类)
资源申请	K8s API (fabric8) 创建Pod	AMRMClient.allocate() 申请Container
Executor启动	K8s创建Pod->容器自启动	NMClient.startContainer()
Executor监控	事件驱动(Watch+Polling)	AMRMClient心跳轮询
资源配置	Feature Step链式(12步)	Client.createContainerLaunchContext()

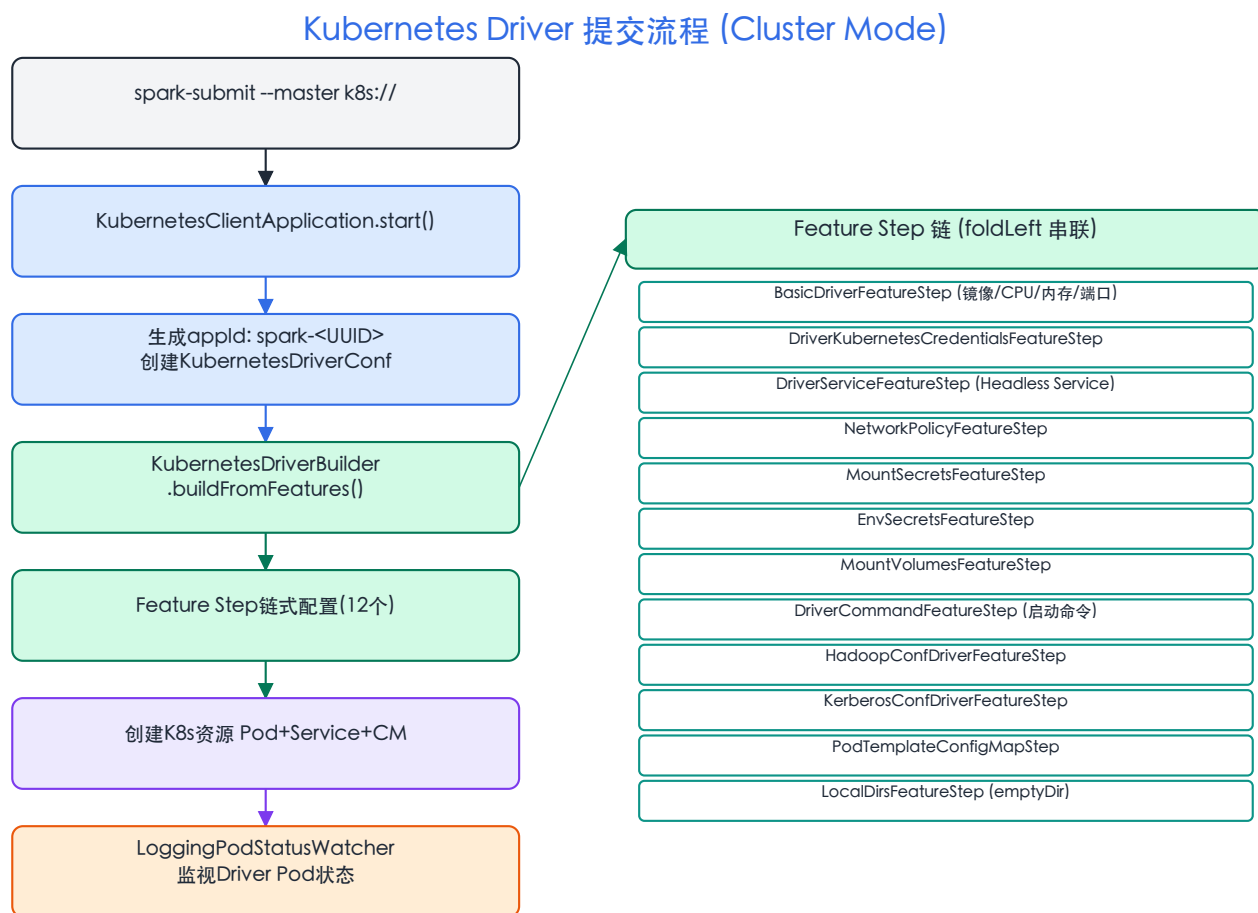
第二章 Kubernetes 资源管理器

Spark on Kubernetes 模块位于 `resource-managers/kubernetes/core`，约40个Scala文件、~6000行代码。采用 Feature Step 链式配置和生产者-消费者事件驱动 两大核心设计模式。

2.1 Driver 提交流程 (Cluster Mode)

Cluster Mode下，`spark-submit` 通过 `KubernetesClientApplication` 将Driver Pod提交到K8s集群。核心入口是 `Client.run()`，使用 `KubernetesDriverBuilder` 构建完整的Driver Pod规格。提交流程为：解析参数 -> 生成`appld(spark-UUID)` -> 创建K8sClient -> 构建Feature Step链 -> 创建Pod/Service/ConfigMap -> 监视Pod状态。

图 2-1: Kubernetes Driver 提交流程



2.2 Feature Step 链式配置

`KubernetesFeatureConfigStep` 是核心trait，定义了4个方法：`configurePod`(修改Pod规格)、`getAdditionalPodSystemProperties`(JVM属性)、`getAdditionalPreKubernetesResources`(Pod前资源)、`getAdditionalKubernetesResources`(Pod后资源)。所有Step通过 `foldLeft`

串联，每步输出是下步输入。用户可通过配置注入自定义Step实现Sidecar注入、自定义Volume等。

```
trait KubernetesFeatureConfigStep {  
  def configurePod(pod: SparkPod): SparkPod  
  def getAdditionalPodSystemProperties(): Map[String, String]  
  def getAdditionalPreKubernetesResources(): Seq[HasMetadata]  
  def getAdditionalKubernetesResources(): Seq[HasMetadata]  
}
```

2.3 Executor Pod 事件驱动模型

Executor Pod管理采用 生产者-消费者模型：事件源(Watch增量+Polling全量)产生Pod状态事件 ->
 ExecutorPodsSnapshotsStoreImpl 聚合为快照(维护currentSnapshot全局视图) ->

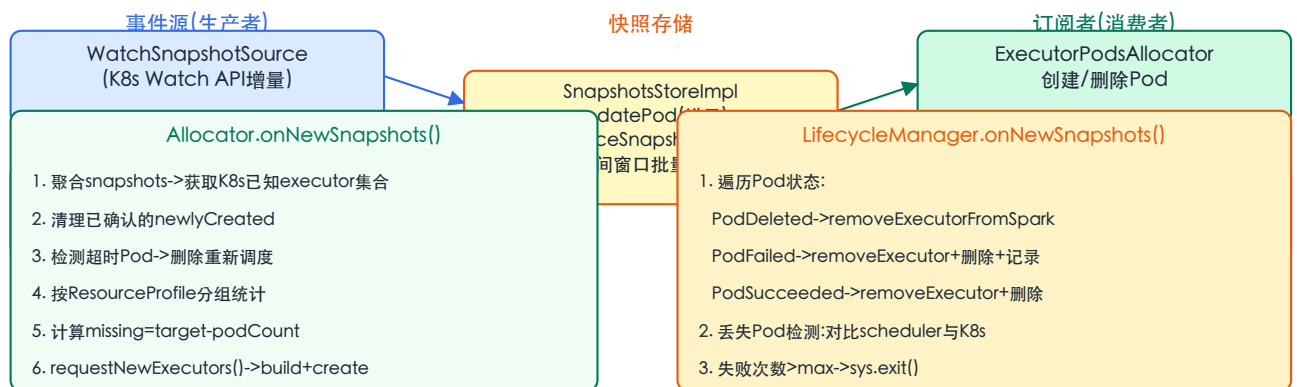
时间窗口批量通知两个消费者。ExecutorPodsAllocator

负责计算missing数并创建/删除Pod；ExecutorPodsLifecycleManager

负责处理Pod终态(Failed/Succeeded/Deleted)并通知Driver移除Executor。

图 2-2: Executor Pod 事件驱动模型

Kubernetes Executor Pod 事件驱动模型



2.4 Pod 分配器策略

策略	配置值	类名	适用场景
直接Pod	direct(默认)	ExecutorPodsAllocator	通用场景,直接创建独立Pod
StatefulSet	statefulset	StatefulSetPodsAllocator	需要稳定网络标识
Deployment	deployment	DeploymentPodsAllocator	需要滚动更新和自愈

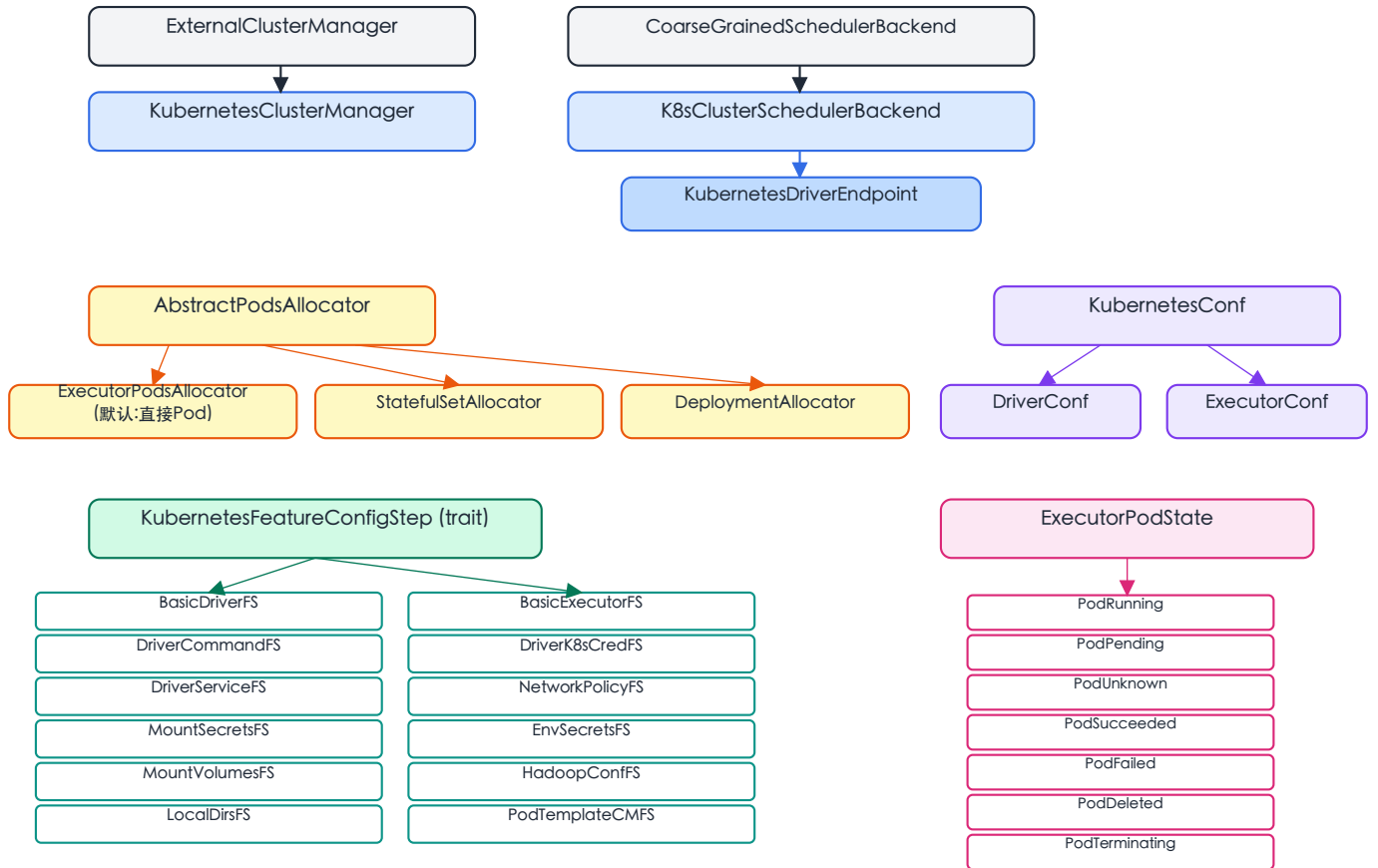
三者均继承

AbstractPodsAllocator，通过配置选择。在Pod创建和删除方式上不同，但共享相同的快照事件处理机制。

2.5 核心类继承体系

图 2-3: Kubernetes模块核心类继承体系

Kubernetes 模块核心类继承体系



核心类说明：

类名	职责	规模
KubernetesClusterManager	SPI入口,创建Scheduler和Backend	~60行
KubernetesClusterSchedulerBackend	调度后端,桥接Spark调度器和K8s API	~400行
KubernetesClientApplication	Cluster Mode提交入口	~260行
KubernetesDriverBuilder	Driver Pod构建器,串联Feature Step链	~110行
KubernetesExecutorBuilder	Executor Pod构建器	~96行
ExecutorPodsAllocator	Pod分配器,核心事件消费者	~630行
ExecutorPodsLifecycleManager	Pod生命周期管理	~340行
ExecutorPodsSnapshotsStoreImpl	快照存储,生产者-消费者中间件	~170行
KubernetesConf	配置抽象基类(Driver/Executor子类)	~336行
KubernetesFeatureConfigStep	核心trait.Pod配置步骤接口	~86行
BasicDriverFeatureStep	Driver核心Feature(镜像/CPU/内存)	~197行
BasicExecutorFeatureStep	Executor核心Feature	~333行

2.6 Shuffle 与 PVC 复用

K8s环境通过 `KubernetesLocalDiskShuffleDataIO` 和 `KubernetesLocalDiskShuffleExecutorComponents` 实现基于PVC的Shuffle数据恢复机制：

- PVC复用: reusePVC=true时,新Executor优先复用已释放PVC,按storageClass和size匹配
- 数据恢复: 扫描本地目录->找data/index/checksum->校验->注册到BlockManager
- 等待复用: waitToReusePVC=true且达maxPVCs上限时等待PVC释放

2.7 插件扩展

插件	类名	功能	触发条件
滚动更新	ExecutorRollPlugin	周期选择executor decommission替换策略:ID/GC_TIME/DURATION/OUTLIER等	rollInterval>0
内存扩容	ExecutorResizePlugin	K8s metrics检测+原地扩容 利用KEP 1287原地Pod更新	resize.enabled=true

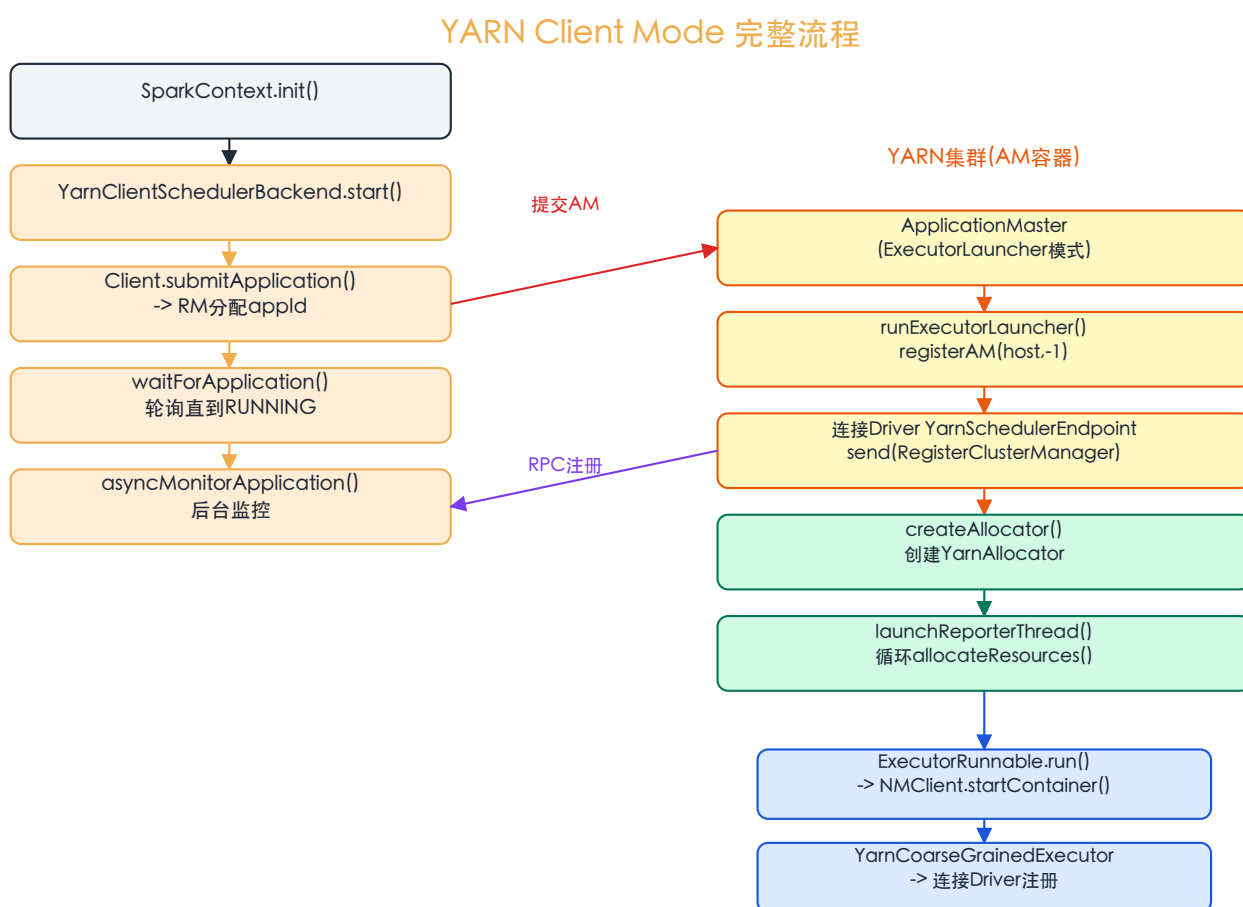
第三章 YARN 资源管理器

Spark on YARN模块位于 resource-managers/yarn，约25个Scala+4个Java文件、~7000行代码。核心文件：Client.scala(76KB/1836行)、YarnAllocator.scala(49KB/1065行)、ApplicationMaster.scala(39KB/977行)。YARN模块的一大特色是支持 Client Mode 和 Cluster Mode 两种部署模式。

3.1 Client Mode 完整流程

Client Mode下Driver运行在用户端机器，AM仅作为 ExecutorLauncher 负责资源分配。YarnClientSchedulerBackend.start()中创建Client对象并submitApplication()，然后waitForApplication()轮询至RUNNING，最后启动MonitorThread后台监控。AM启动后通过RPC向Driver发送RegisterClusterManager注册自身。

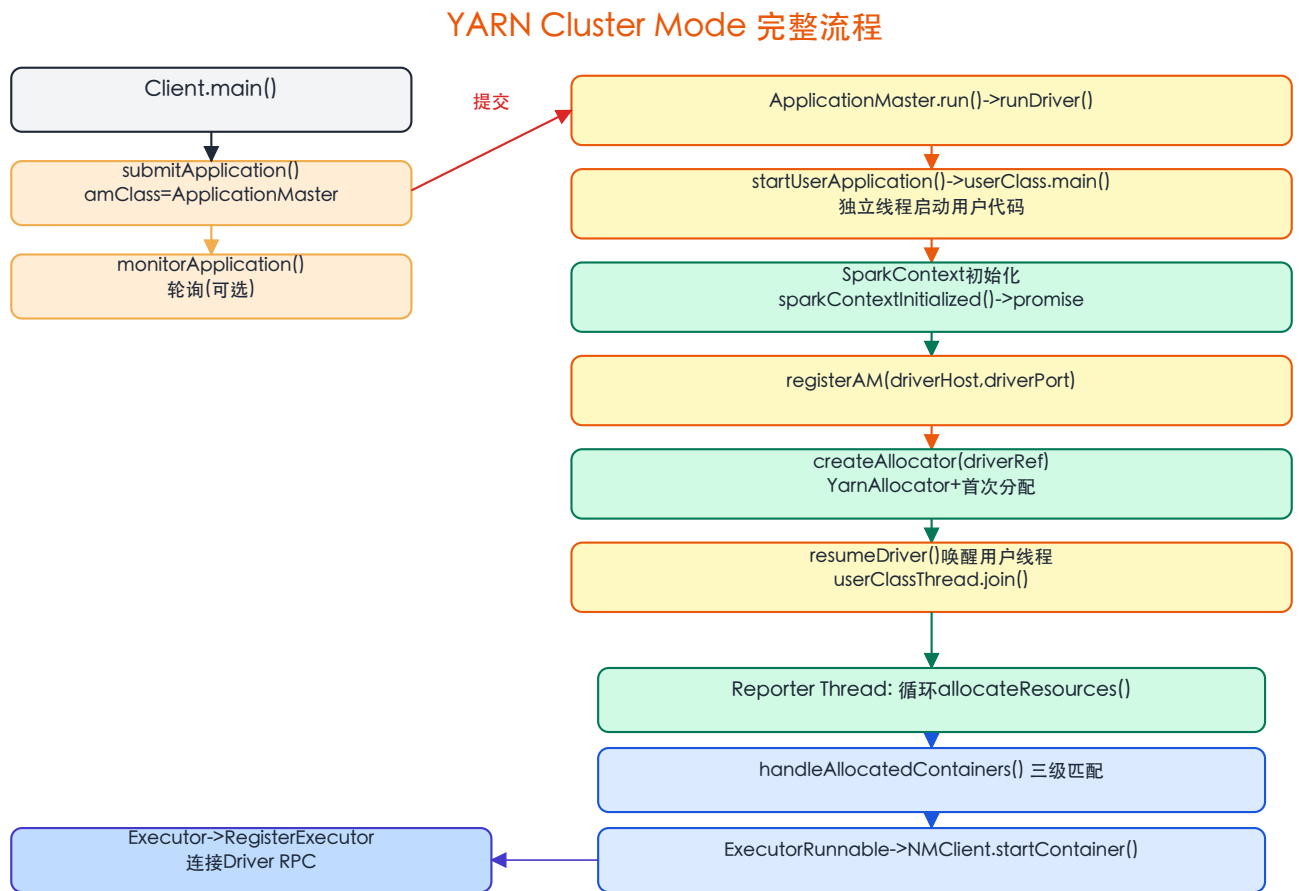
图 3-1: YARN Client Mode 完整流程



3.2 Cluster Mode 完整流程

Cluster Mode下Driver运行在AM容器内。AM的runDriver()方法在独立线程中通过反射调用用户主类，SparkContext初始化时调用sparkContextInitialized()设置Promise。AM等待Promise完成后registerAM()、createAllocator()、最后resumeDriver()唤醒用户线程继续执行。

图 3-2: YARN Cluster Mode 完整流程



3.2.1 Client vs Cluster 关键区别

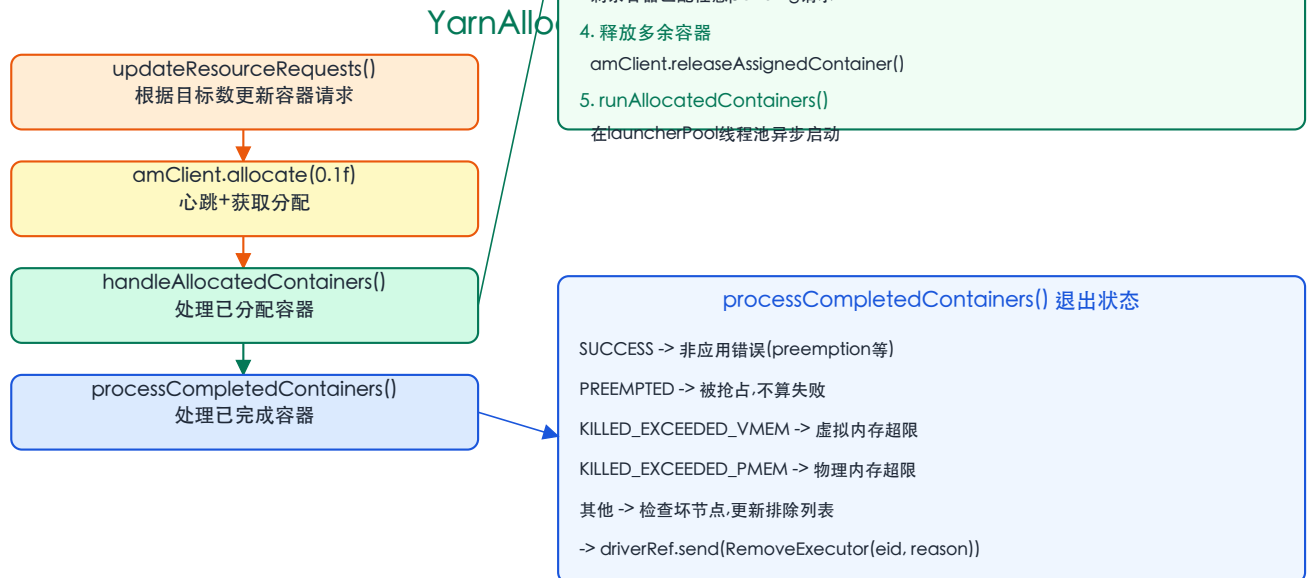
维度	Client Mode	Cluster Mode
Driver位置	客户端机器	AM容器内
AM角色	仅资源分配(ExecutorLauncher)	Driver+资源分配
AM内存	spark.yarn.am.memory	spark.driver.memory
AM核数	spark.yarn.am.cores	spark.driver.cores
TaskScheduler	YarnScheduler	YarnClusterScheduler
SchedulerBackend	YarnClientSchedulerBackend	YarnClusterSchedulerBackend
attemptId	None	Some(attemptId)

AM注册端口	-1(不监听)	Driver实际端口
--------	---------	------------

3.3 YarnAllocator 资源分配详解

YarnAllocator 是YARN资源管理的核心引擎(1065行)
机制(用ResourceProfile ID作Priority值)区分不同资源
resourceRequests(计算missing并发送请求)、amClient
匹配+启动)、processCompletedContainers(退出)

图 3-3: YarnAllocator



关键数据结构：

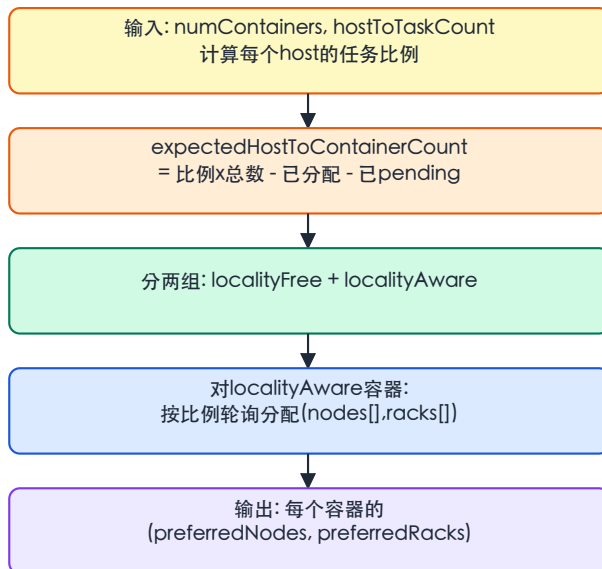
数据结构	类型	用途
allocatedHostToContainersMapPerRPId	Map[Int, Map[String, Set]]	每个RP的host->容器映射
executorIdToContainer	Map[String, Container]	executorId->Container映射
rpIdToYarnResource	Map[Int, Resource]	RP ID->YARN Resource映射
targetNumExecutorsPerRPId	Map[Int, Int]	每个RP的目标executor数
failedExecutorsTimeStamps	Map[String, Long]	失败executor时间戳
pendingLossReasonRequests	Map[String, ...]	待处理的丢失原因请求

3.4 本地性匹配策略

LocalityPreferredContainerPlacementStrategy 实现了智能
根据各host的任务数量比例计算每个新容器应偏好的节点和机架。算
unt -> 分localityFree和localityAware两组 -> 对localityAware

图 3-4: 本地性偏好容器放

YARN 本地性偏好



示例

假设4个host的任务分布:
host1:30 host2:30 host3:20 host4:10

请求15个容器时按比例:
host1: 33% -> 5个
host2: 33% -> 5个
host3: 22% -> 3个
host4: 11% -> 2个

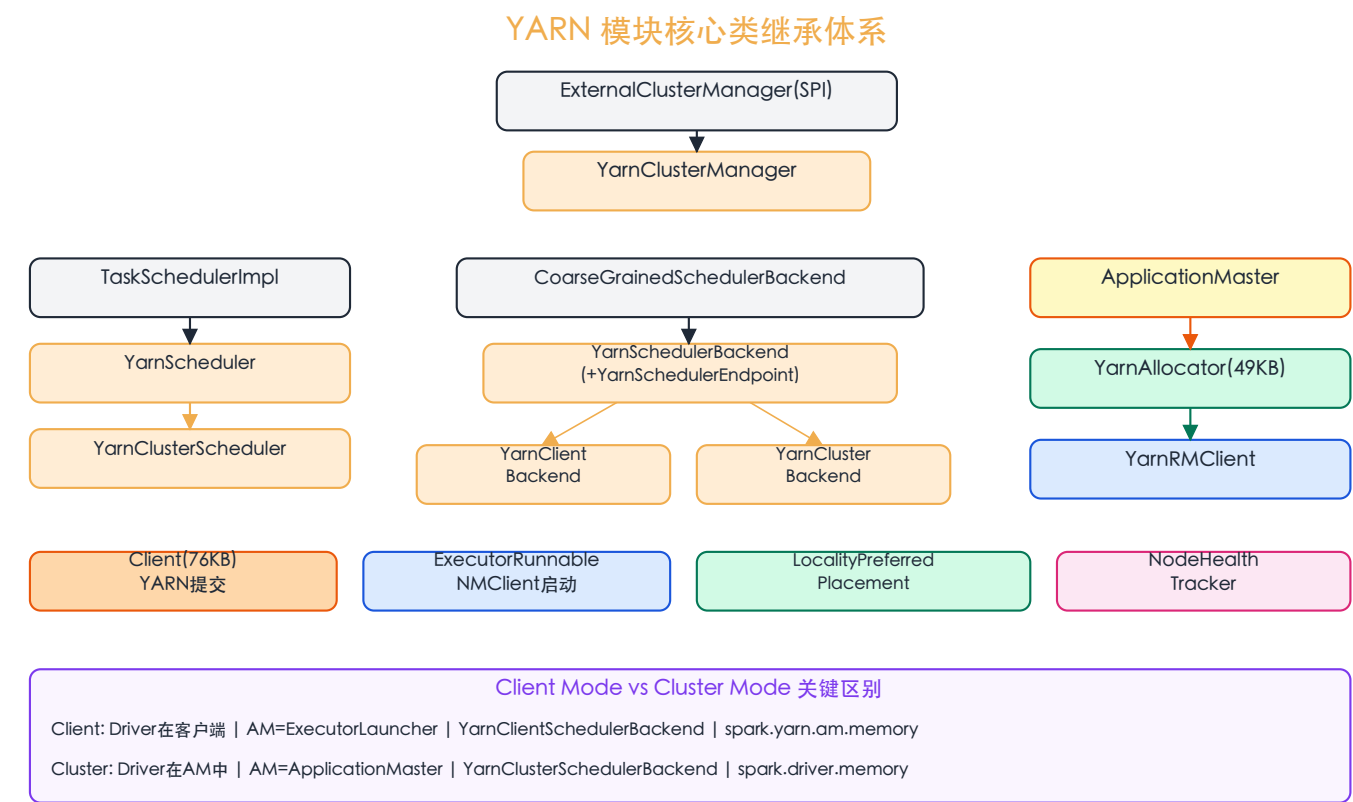
轮询分配偏好:
容器1 -> prefer host1
容器2 -> prefer host2
容器3 -> prefer host1
容器4 -> prefer host2
...(递减直到ratio为0)

三级容器匹配流程：

- 第1级 NODE_LOCAL: 按主机名精确匹配,容器与请求在同一节点
- 第2级 RACK_LOCAL: 解析机架信息(独立线程避免阻塞),按机架匹配
- 第3级 OFF_RACK (ANY): 剩余容器匹配任意待处理请求
- 释放多余: 无法匹配的容器通过releaseAssignedContainer释放回RM

3.5 核心类继承体系

图 3-5: YARN模块核心类继承体系



核心类说明：

类名	职责	规模
YarnClusterManager	SPI入口,根据deployMode创建不同组件	~57行
YarnSchedulerBackend	公共抽象基类,含YarnSchedulerEndpoint	~412行
YarnClientSchedulerBackend	Client模式后端,提交应用+MonitorThread	~190行
YarnClusterSchedulerBackend	Cluster模式后端,从AM获取attemptId	~51行
Client	YARN提交客户端(最大文件)	~1836行
ApplicationMaster	AM主逻辑,runDriver/runExecutorLauncher	~977行
YarnAllocator	资源分配器,容器请求/分配/启动/回收	~1065行
ExecutorRunnable	NMClient启动Executor容器	~232行
YarnRMClient	AMRMClient封装	~154行
LocalityPreferredContainerPlacementStrategy	容器放置本地性策略	~231行
YarnAllocatorNodeHealthTracker	节点健康追踪与排除管理	~153行
ResourceRequestHelper	自定义YARN资源(GPU/FPGA)映射	~190行

3.6 AM 与 RM 交互机制

ApplicationMaster 通过 YarnRMClient 与YARN RM交互，使用Hadoop的AMRMClient实现心跳、资源请求和注销：

交互阶段	方法	说明
注册	amClient.registerApplicationMaster()	传入host/port/trackingUrl
心跳	amClient.allocate(0.1f)	返回AllocateResponse(新分配+已完成+节点变更)
请求容器	amClient.addContainerRequest()	带优先级、节点/机架偏好的ContainerRequest
取消请求	amClient.removeContainerRequest()	取消pending中的请求
释放容器	amClient.releaseAssignedContainer()	释放不需要的容器
节点排除	amClient.updateBlacklist()	添加/移除排除节点列表
注销	amClient.unregisterApplicationMaster()	传入最终状态和诊断信息

Reporter Thread 心跳机制：

AM内部的Reporter Thread以自适应间隔循环调用allocateResources()：有pending请求时使用短间隔(指数退避)，无pending时使用heartbeatInterval。每次循环还检查executor失败数是否超限、是否所有节点被排除。

YarnSchedulerEndpoint RPC通信：

Driver端的 YarnSchedulerEndpoint 是AM与Driver的RPC桥梁。AM启动时发送RegisterClusterManager注册自身引用。之后Driver的RequestExecutors/KillExecutors请求通过此端点转发到AM的AMEndpoint，AMEndpoint再调用YarnAllocator的对应方法。YarnDriverEndpoint重写了onDisconnected方法，在Executor断连时先向AM询问退出原因(如YARN抢占)，避免误判任务失败。

第四章 Kubernetes vs YARN 架构对比

图 4-1：两种资源管理器架构对比



详细对比分析：

对比维度	Kubernetes	YARN
资源抽象	Pod(可包含多容器)	Container(单进程)
资源申请方式	直接通过K8s API创建Pod	通过AMRMClient心跳时携带请求
资源配置模式	Feature Step链式(12步,可扩展)	单一方法createContainerLaunchContext
事件监控	Watch(实时增量)+Polling(全量同步) 生产者-消费者快照模型	AMRMClient.allocate()返回已完成列表 心跳轮询模型
本地性策略	无(K8s自身调度)	三级匹配:HOST->RACK->ANY LocalityPreferredPlacement
Executor注册	GenerateExecID(K8s为Pod分配ID) 再RegisterExecutor	直接RegisterExecutor (ID由Driver分配)
部署模式	仅Cluster Mode	Client Mode + Cluster Mode
弹性伸缩	3种Pod分配器 (Direct/StatefulSet/Deployment)	AMRMClient容器请求增减
故障恢复	PVC复用+Shuffle数据恢复	容器重试+AM重试
安全模型	K8s RBAC+ServiceAccount	Kerberos+Token续期+AMIPFilter
插件扩展	ExecutorRoll/Resize Plugin	无内置插件机制
代码规模	~40文件, ~6000行	~29文件, ~7000行

第五章 核心设计精髓总结

5.1 ExternalClusterManager SPI 可插拔架构

Spark通过Java SPI机制加载资源管理器实现，核心接口 ExternalClusterManager 只需实现4个方法：canCreate(判断master URL)、createTaskScheduler、createSchedulerBackend、initialize。这使得添加新的资源管理器只需编写一个实现类并注册到META-INF/services即可，完全不需要修改Spark核心代码。

5.2 Feature Step 链式模式 (K8s)

K8s模块的Feature Step链式模式是一种优雅的管道-过滤器(Pipeline-Filter)设计模式应用。每个Feature只关注自己的配置维度(如Secret、Volume、网络策略)，通过foldLeft串联实现关注点分离。用户可以通过配置注入自定义Step，实现零代码修改的Pod定制。这种设计既保证了内置功能的完整性，又提供了极强的扩展性。

5.3 生产者-消费者事件驱动 (K8s)

K8s模块的Pod管理采用了经典的生产者-消费者模型，通过 ExecutorPodsSnapshotsStoreImpl 解耦事件源和处理器。Watch提供实时增量更新，Polling提供定期全量同步(防止Watch事件丢失)，两者互补保证一致性。时间窗口批量通知机制减少了不必要的处理频率，提高了系统稳定性。

5.4 AM双模式架构 (YARN)

YARN模块通过同一个 ApplicationMaster 类支持两种截然不同的运行模式：Cluster Mode下AM既是Driver又是资源管理器；Client Mode下AM仅作为ExecutorLauncher。通过 isClusterMode = (args.userClass != null) 一行代码区分，然后分别进入runDriver()和runExecutorLauncher()两条执行路径。这种设计避免了代码重复，最大化了复用。

5.5 三级本地性匹配 (YARN)

YARN模块的容器分配实现了精细的本地性优化：首先通过 LocalityPreferredContainerPlacementStrategy 按任务分布比例计算容器偏好，然后在handleAllocatedContainers()中按 HOST->RACK->ANY 三级匹配。这种两阶段本地性策略既充分利用了数据本地性，又避免了资源浪费。

5.6 CoarseGrainedSchedulerBackend 统一基类

K8s和YARN的调度后端都继承自 CoarseGrainedSchedulerBackend，共享Executor注册、任务分发、资源请求等核心逻辑。各资源管理器只需关注自己平台特有的资源申请和生命周期管理，极大降低了开发和维护成本。DriverEndpoint RPC通信框架为两种部署模式提供了统一的Executor管理接口。